

```

1  """Lecture 8 demonstrations: analog prototypes, bilinear transform, and IIR
  preprocessing."""
2  from __future__ import annotations
3
4  from pathlib import Path
5  import numpy as np
6  import matplotlib.pyplot as plt
7  from scipy import signal
8
9  OUT = Path(__file__).resolve().parent / "assets"
10 OUT.mkdir(exist_ok=True)
11
12
13 def plot_analog_prototypes() -> None:
14     N = 5
15     wc = 1.0
16     designs = {
17         "Butterworth": signal.butter(N, wc, analog=True, output="ba"),
18         "Chebyshev I (1 dB ripple)": signal.cheby1(N, 1, wc, analog=True, output="ba"),
19         "Chebyshev II (40 dB stop)": signal.cheby2(N, 40, 1.6 * wc, analog=True, output="ba"),
20         "Elliptic (1/40 dB)": signal.ellip(N, 1, 40, wc, analog=True, output="ba"),
21     }
22     Om = np.logspace(-1.2, 1.1, 1500)
23     fig, ax = plt.subplots(figsize=(8.8, 5.0))
24     for name, (b, a) in designs.items():
25         _, H = signal.freqs(b, a, worN=Om)
26         ax.semilogx(Om, 20 * np.log10(np.maximum(np.abs(H), 1e-8)), label=name)
27     ax.axvline(wc, linestyle="--", linewidth=1.1)
28     ax.set_ylim(-90, 5)
29     ax.set_title("Classical analog lowpass prototypes: same goal, different tradeoffs")
30     ax.set_xlabel(r"Analog frequency $\Omega$ (rad/s, normalized)")
31     ax.set_ylabel("Magnitude (dB)")
32     ax.grid(True, which="both", alpha=0.25)
33     ax.legend(fontsize=8)
34     fig.tight_layout()
35     fig.savefig(OUT / "iir_analog_prototypes.png", dpi=180)
36     plt.close(fig)
37
38
39 def plot_bilinear_mapping() -> None:
40     T = 0.2
41     # Representative stable analog poles.
42     s = np.array([-0.5 + 2j, -1 + 1j, -2 + 0j, -3 - 2j, -0.3 - 3j])
43     z = (1 + s * T / 2) / (1 - s * T / 2)
44
45     theta = np.linspace(0, 2 * np.pi, 500)
46     fig, ax = plt.subplots(figsize=(6.4, 6.0))
47     ax.plot(np.cos(theta), np.sin(theta), linewidth=1.3, label="unit circle")
48     ax.scatter(z.real, z.imag, s=55, label="mapped stable poles")
49     for sk, zk in zip(s, z):
50         ax.annotate(f"{{sk.real:.1f}}{{sk.imag:+.1f}}j", (zk.real, zk.imag),
51                   xytext=(5, 5), textcoords="offset points", fontsize=8)
52     ax.axhline(0, linewidth=0.8)
53     ax.axvline(0, linewidth=0.8)
54     ax.set_aspect("equal", adjustable="box")
55     ax.set_xlim(-1.15, 1.15)
56     ax.set_ylim(-1.15, 1.15)
57     ax.set_title("Bilinear transform maps the analog LHP inside $|z|<1$")
58     ax.set_xlabel("Re{$z$}")
59     ax.set_ylabel("Im{$z$}")
60     ax.grid(True, alpha=0.25)
61     ax.legend(loc="lower left")
62     fig.tight_layout()
63     fig.savefig(OUT / "iir_bilinear_pole_map.png", dpi=180)
64     plt.close(fig)
65

```

```

66
67 def plot_frequency_warping() -> None:
68     omega = np.linspace(0, 0.985 * np.pi, 1000)
69     T = 1.0
70     Omega = (2 / T) * np.tan(omega / 2)
71     fig, ax = plt.subplots(figsize=(8.8, 5.0))
72     ax.plot(omega / np.pi, Omega, linewidth=2)
73     ax.plot(omega / np.pi, omega / T, "--", linewidth=1.2,
74             label="small-frequency linear approximation")
75     ax.set_ylim(0, 20)
76     ax.set_xlim(0, 1)
77     ax.set_title("Bilinear transform frequency warping")
78     ax.set_xlabel(r"Digital frequency $\omega/\pi$")
79     ax.set_ylabel(r"Analog frequency $\Omega$ for $T=1$")
80     ax.grid(True, alpha=0.25)
81     ax.legend()
82     fig.tight_layout()
83     fig.savefig(OUT / "iir_frequency_warping.png", dpi=180)
84     plt.close(fig)
85
86
87 def prewarp_example() -> tuple[float, float]:
88     T = 1.0
89     omega_c = 0.4 * np.pi
90     Omega_c = (2 / T) * np.tan(omega_c / 2)
91
92     # Analog Butterworth prototype at the prewarped cutoff, then bilinear transform.
93     b_a, a_a = signal.butter(4, Omega_c, analog=True, output="ba")
94     b_z, a_z = signal.bilinear(b_a, a_a, fs=1 / T)
95     w, H = signal.freqz(b_z, a_z, worN=4096)
96
97     # Find approximate -3 dB location.
98     db = 20 * np.log10(np.maximum(np.abs(H), 1e-12))
99     idx = np.argmin(np.abs(db + 3.0103))
100    omega_meas = w[idx]
101
102    fig, ax = plt.subplots(figsize=(8.8, 5.0))
103    ax.plot(w / np.pi, db, linewidth=1.8)
104    ax.axvline(omega_c / np.pi, linestyle="--", linewidth=1.2,
105              label="desired digital cutoff")
106    ax.axhline(-3.0103, linestyle=":", linewidth=1.1)
107    ax.set_ylim(-80, 3)
108    ax.set_xlim(0, 1)
109    ax.set_title("Prewarping places the Butterworth cutoff at the desired digital
110    frequency")
111    ax.set_xlabel(r"Normalized digital frequency $\omega/\pi$")
112    ax.set_ylabel("Magnitude (dB)")
113    ax.grid(True, alpha=0.25)
114    ax.legend()
115    fig.tight_layout()
116    fig.savefig(OUT / "iir_prewarp_design.png", dpi=180)
117    plt.close(fig)
118    return Omega_c, omega_meas
119
120 def plot_iir_phase() -> None:
121     b, a = signal.butter(5, 0.25)
122     w, H = signal.freqz(b, a, worN=2048)
123     phase = np.unwrap(np.angle(H))
124     gd_w, gd = signal.group_delay((b, a), w=1024)
125
126     fig, ax = plt.subplots(figsize=(8.8, 5.0))
127     ax.plot(w / np.pi, phase, linewidth=1.8)
128     ax.set_title("A typical IIR lowpass has nonlinear phase")
129     ax.set_xlabel(r"Normalized frequency $\omega/\pi$")
130     ax.set_ylabel("Unwrapped phase (rad)")
131     ax.grid(True, alpha=0.25)

```

```

132 fig.tight_layout()
133 fig.savefig(OUT / "iir_nonlinear_phase.png", dpi=180)
134 plt.close(fig)
135
136 fig, ax = plt.subplots(figsize=(8.8, 5.0))
137 ax.plot(gd_w / np.pi, gd, linewidth=1.8)
138 ax.set_xlim(0, 0.55)
139 ax.set_ylim(0, np.nanpercentile(gd[np.isfinite(gd)], 95) * 1.2)
140 ax.set_title("Nonlinear phase appears as frequency-dependent group delay")
141 ax.set_xlabel(r"Normalized frequency  $\omega/\pi$ ")
142 ax.set_ylabel("Group delay (samples)")
143 ax.grid(True, alpha=0.25)
144 fig.tight_layout()
145 fig.savefig(OUT / "iir_group_delay.png", dpi=180)
146 plt.close(fig)
147
148
149 def plot_filtfilt_demo() -> None:
150     rng = np.random.default_rng(11)
151     fs = 1000.0
152     t = np.arange(0, 1.0, 1 / fs)
153     # A transient plus useful 12-Hz oscillation and high-frequency interference.
154     clean = np.exp(-((t - 0.42) / 0.018) ** 2) + 0.35 * np.sin(2 * np.pi * 12 * t)
155     noisy = clean + 0.25 * np.sin(2 * np.pi * 150 * t) + 0.07 * rng.standard_normal(t.size)
156
157     sos = signal.butter(4, 40, btype="low", fs=fs, output="sos")
158     causal = signal.sosfilt(sos, noisy)
159     zero_phase = signal.sosfiltfilt(sos, noisy)
160
161     fig, ax = plt.subplots(figsize=(9.2, 5.0))
162     mask = (t > 0.30) & (t < 0.56)
163     ax.plot(t[mask], noisy[mask], label="noisy input", linewidth=0.9, alpha=0.7)
164     ax.plot(t[mask], clean[mask], label="reference", linewidth=1.3)
165     ax.plot(t[mask], causal[mask], label="causal IIR", linewidth=1.6)
166     ax.plot(t[mask], zero_phase[mask], label="forward-backward (zero phase)", linewidth=1.8)
167     ax.set_title("Offline forward-backward filtering removes phase delay but is noncausal")
168     ax.set_xlabel("Time (s)")
169     ax.set_ylabel("Amplitude")
170     ax.grid(True, alpha=0.25)
171     ax.legend(ncol=2)
172     fig.tight_layout()
173     fig.savefig(OUT / "iir_filtfilt_transient.png", dpi=180)
174     plt.close(fig)
175
176     w, H = signal.sosfreqz(sos, worN=2048, fs=fs)
177     fig, ax = plt.subplots(figsize=(8.8, 5.0))
178     one_pass_db = 20 * np.log10(np.maximum(np.abs(H), 1e-10))
179     two_pass_db = 20 * np.log10(np.maximum(np.abs(H) ** 2, 1e-10))
180     ax.plot(w, one_pass_db, label="one pass:  $|H|$ ", linewidth=1.8)
181     ax.plot(w, two_pass_db, label="forward-backward:  $|H|^2$ ", linewidth=1.8)
182     ax.set_xlim(0, 180)
183     ax.set_ylim(-100, 3)
184     ax.set_title("Forward-backward filtering squares the magnitude response")
185     ax.set_xlabel("Frequency (Hz)")
186     ax.set_ylabel("Magnitude (dB)")
187     ax.grid(True, alpha=0.25)
188     ax.legend()
189     fig.tight_layout()
190     fig.savefig(OUT / "iir_filtfilt_magnitude.png", dpi=180)
191     plt.close(fig)
192
193
194 def main() -> None:
195     plot_analog_prototypes()

```

```
196     plot_bilinear_mapping()
197     plot_frequency_warping()
198     Omega_c, omega_meas = prewarp_example()
199     plot_iir_phase()
200     plot_filtfilt_demo()
201     print(f"Prewarped analog cutoff for omega_c=0.4*pi, T=1: {Omega_c:.6f} rad/s")
202     print(f"Measured digital -3 dB cutoff: {omega_meas/np.pi:.4f} * pi rad/sample")
203
204
205 if __name__ == "__main__":
206     main()
207
```